

Tammi-peli / Chinook

Joni Makkonen, AB7464
Marika Matalamäki, AA4495
Pauliina Mäkinen, AA4972

Harjoitustyö
Marraskuu 2021
Tekniikan ala
Insinööri (AMK), Tieto- ja viestintätekniikka
Tietorakenteet ja algoritmit TTC1050-300

Sisältö

1	Johdanto	2
1.1	Pelin säännöt lyhyesti	3
2	Min-Max	4
2.1	Käyttö ja toimintaperiaate.....	4
3	Chinook.....	8
3.1	Chinookin historia.....	8
3.2	Chinookin käyttämät algoritmit.....	10
3.2.1	Lopputietokanta (Backward search / endgame database).....	10
3.2.2	Pelin laskuosa (Forward search).....	10
3.3	Chinookin toiminta kuvana.....	13
4	Algoritmien vertailu.....	13
4.1	Harjoitustyössä käytetty checkers AI	13
4.2	Min-max vastaan Chinook	14
5	Yhteenveto / pohdinta	15
	Lähteet	17

Kuviot

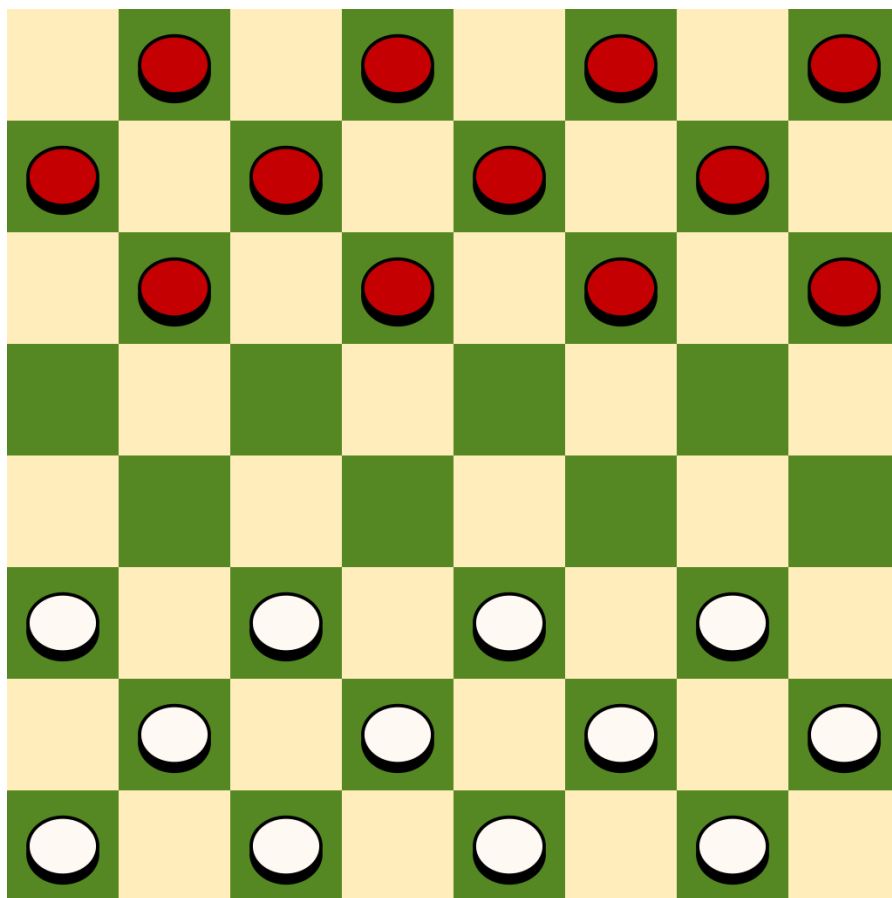
Kuvio 1 8x8 tammipelilauta.....	2
Kuvio 2 Tammen mahdolliset pelitilanteet, jos pelaajilla on 12 pelinappulaa	3
Kuvio 3 Aloitustilanne, pieni ympyrä kuvaa mahdollisia siirtoja.....	5
Kuvio 4 Esimerkki siirroista.....	6
Kuvio 5 Esimerkki siirtojen pisteytyksestä	7
Kuvio 6 Tallennetut aloitukset	11
Kuvio 7 Chinookin forward ja backward search.....	12
Kuvio 8 Chinookin toiminta lyhyesti kuvana	13
Kuvio 9 min-max-muutos	14
Kuvio 10 Chinookin käyttöliittymä	15

Taulukot

Taulukko 1 Endgame-tietokannan kasvu 1989–2007	9
--	---

1 Johdanto

Tammi-pelin historiaa ei varmasti tiedetä, mutta sen arvellaan juontavansa juurensa muinaiseen Egyptiin ja ensimmäisiin versioihin shakista. Varmistetut tiedot tammen pelaamisesta on tehty Ranskassa noin 1000-luvulla. Tammesta on monia eri variaatioita, tavallisin länsimaalaisessa kulttuurissa on 8x8 pelilaudalla pelattava kaksinpeli, jossa molemmilla pelaajilla on käytössään 12 pelinappulaa. (Tammipeli, Wikipedia, 2021)



Kuvio 1 8x8 tammipelilauta

1.1 Pelin säännöt lyhyesti

Pelilaudalla on kahden värisiä ruutuja, joista käytetään vain yhtä väriä (yleensä mustia). Nappuloita siirretään askel kerrallaan vuorotellen etuviistoon tavoitteena syödä toisen nappulat ”hyppäämällä niiden yli”. Pelissä on pakko syödä useampi vihollisen pelinappula kerralla, jos se on mahdollista. Pelinappula muuttuu kuninkaaksi, joka voi liikkua myös takaviistoon, kun pelinappula saavuttaa pelaajasta itsestä katsottuna kauimmaisen rivin.

Peli päättyy, kun jommankumman pelaajan kaikki nappulat on syöty, syntyy pattitilanne tai jompikumpi pelaajista luovuttaa tai ehdotta tasapeliä.

Pieces	Number of positions
1	120
2	6,972
3	261,224
4	7,092,774
5	148,688,232
6	2,503,611,964
7	34,779,531,480
8	406,309,208,481
9	4,048,627,642,976
10	34,778,882,769,216
Total 1–10	39,271,258,813,439
11	259,669,578,902,016
12	1,695,618,078,654,976
13	9,726,900,031,328,256
14	49,134,911,067,979,776
15	218,511,510,918,189,056
16	852,888,183,557,922,816
17	2,905,162,728,973,680,640
18	8,568,043,414,939,516,928
19	21,661,954,506,100,113,408
20	46,352,957,062,510,379,008
21	82,459,728,874,435,248,128
22	118,435,747,136,817,856,512
23	129,406,908,049,181,900,800
24	90,072,726,844,888,186,880
Total 1–24	500,995,484,682,338,672,639

Kuvio 2 Tammen mahdolliset pelitilanteet, jos pelaajilla on 12 pelinappulaa

Pelilaudalla on yksi nappula; se voi olla musta, tai valkoinen, normaali (28 mahdollista ruutua) tai kuningas (32 ruutua) eli erilaisia variaatioita on 120kpl.

2 Min-Max

Minimax-algoritmi on tunnetuin strategia kahden pelaajan nollasummapeliin. Tämän todisti John von Neumann vuonna 1928. Minimax on strategia, jolla minimoidaan aina suurin mahdollinen tappio, joka voi aiheutua pelaajan valinnasta. (American Psychological Association, Game theory, 1998.)

Nimi "minimax" tulee tappion minimoinnista, kun vastustaja valitsee strategian, joka antaa suurimman tappion. Minimax on hyödyllinen analysoitaessa ensimmäisen pelaajan päätöksiä sekä silloin kun pelaajat liikkuvat peräkkäin sekä silloin kun pelaajat liikkuvat samanaikaisesti. Näistä jälkimmäisessä tapauksessa minimax voi antaa pelin Nash-tasapainon, jos jotkut lisäehdot ovat myös voimassa.

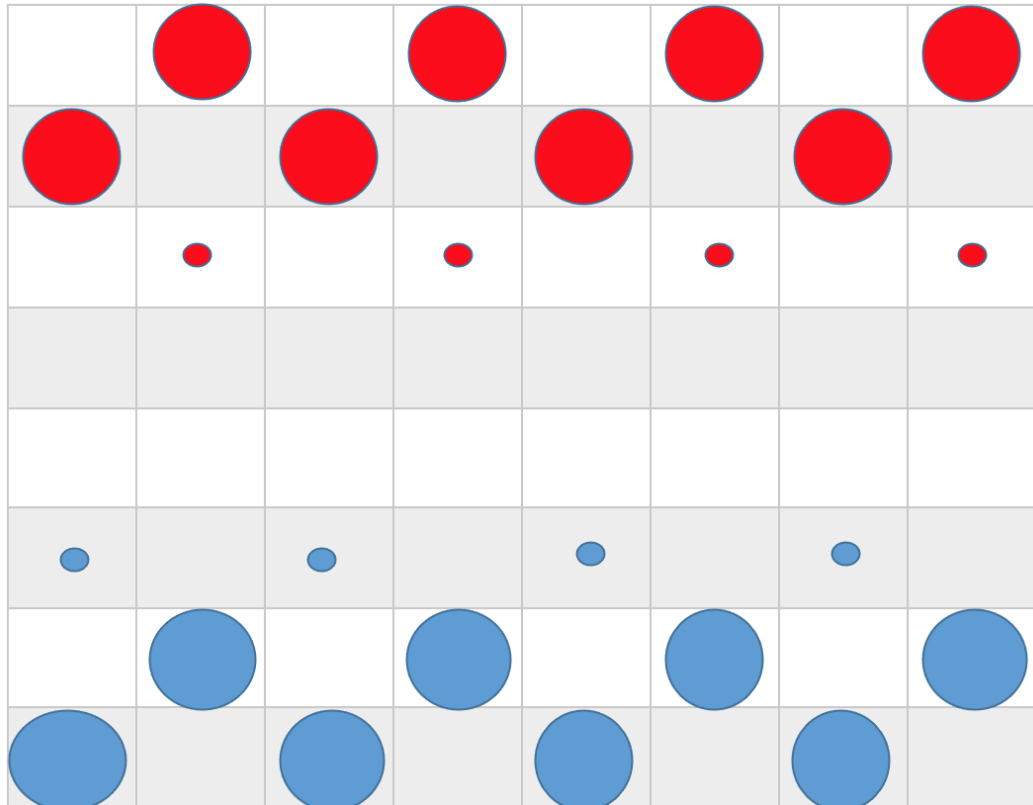
Minimax on hyödyllinen myös kombinatorisissa peleissä, joissa jokaiselle asemalle on määritetty voitto. Yksinkertaisin esimerkki on antaa positiivinen arvo voittoasemalle ja negatiivinen arvo häviävälle, mutta koska tätä on vaikea laskea kaikille paitsi yksinkertaisimmille peleille, väliarviointit (jotka on valittu nimenomaan kyseiselle pelille) ovat yleensä tarpeellista. Tässä yhteydessä ensimmäisen pelaajan tavoitteena on maksimoida paikan arviointi ja toisen pelaajan tavoitteena on minimoida paikan arviointi, joten minimimax-sääntö pätee. Pohjimmiltaan tämä on se tapa, jolla tietokoneet lähestyvät pelejä, kuten shakki ja go, vaikka useita laskennallisia parannuksia on mahdollista tehdä minimaxin toteutukseen. (American Psychological Association, 2021.)

2.1 Käyttö ja toimintaperiaate

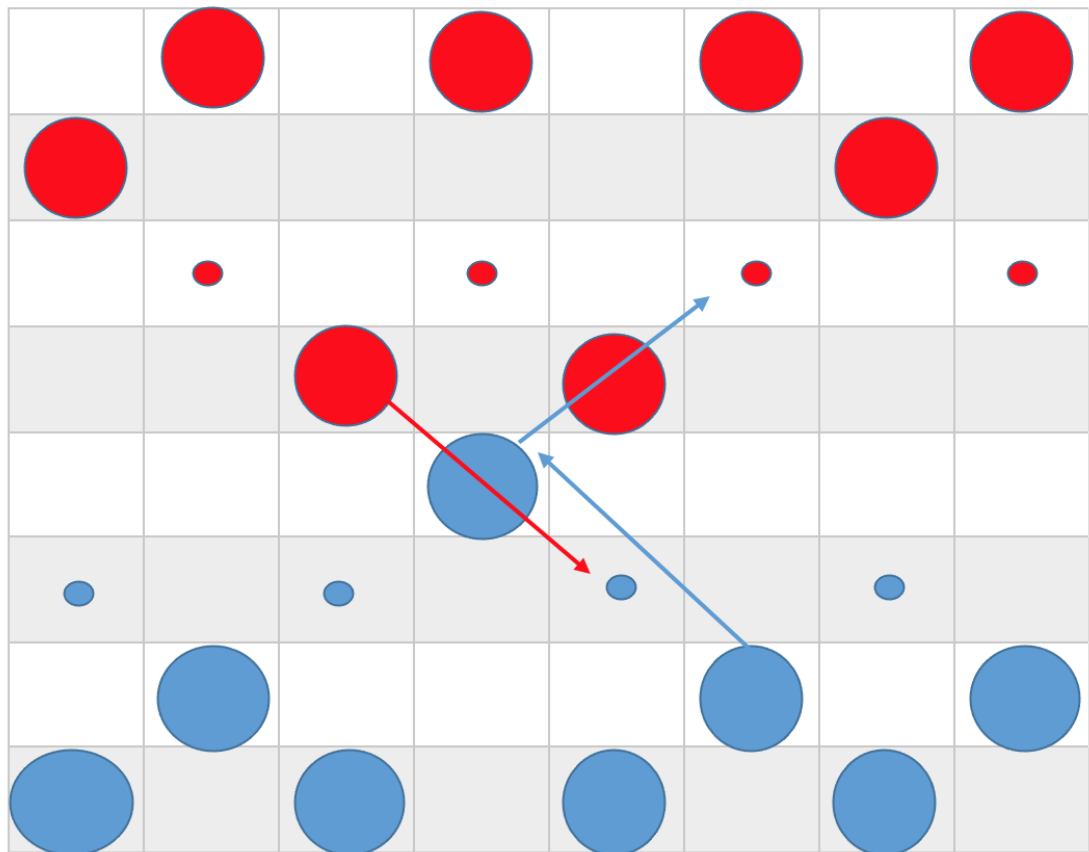
Useiden päätösmuuttujien maksimiarvo pyritään minimoimaan Minimax-ongelmalla. Pahimmassa tapauksessa (suurin tappio) sitä käytetään joskus minimoimaan mahdolliset menetykset. Maksimiongelma maksimoi minimiarvon. Maksimiarvoa käytetään maksimoimaan vähimmäistavoite (kuten voitto tai tuotto) kaikissa mahdollisissa skenaarioissa. (John, D. Hedengren. 2020.)

Minimax on peliteoriassa tai päätöksenteossa oleva taktiikka, jossa yksilöt mm. yrittävät minimoida omat maksimitappionsa. (American Psychological Association,2021.)

Maksimistrategia on peliteoriassa tai päätöksenteossa taktiikka, jossa yksilö valitsee parhaan mahdollisen tuloksen huonoimpien joukosta. (American Psychological Association,2021.)



Kuvio 3 Aloitustilanne, pieni ympyrä kuvaa mahdollisia siirtoja



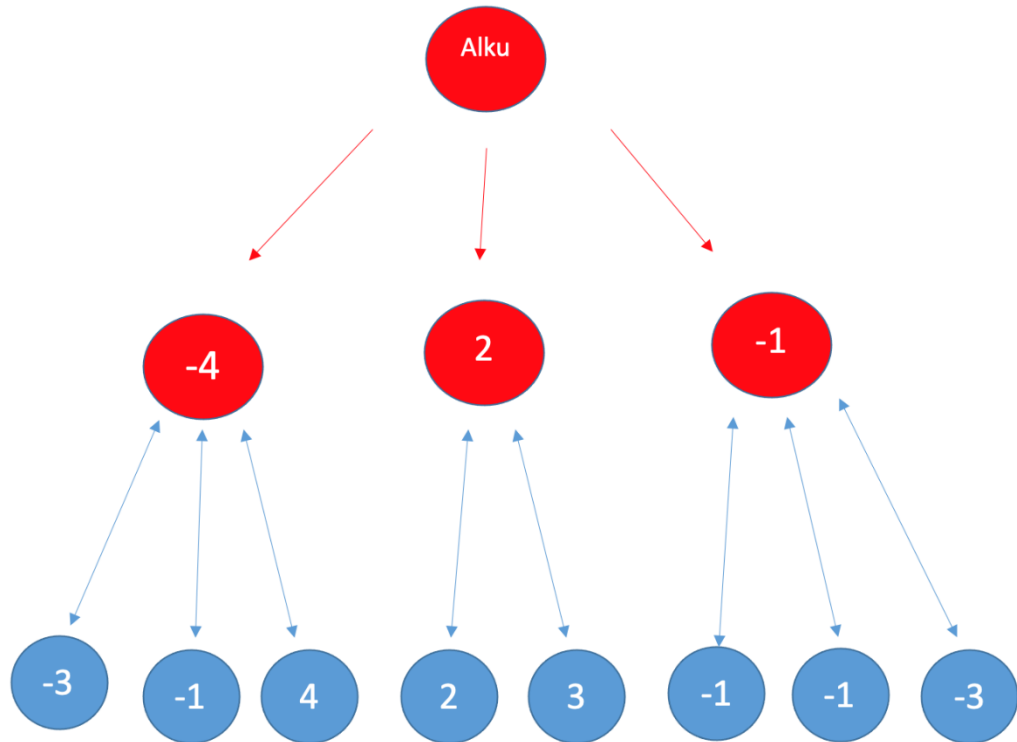
Kuvio 4 Esimerkki siirroista

Esimerkkitalanteessa punainen pystyisi ensin syömään sinisen (punainen nuoli), mutta sen jälkeen sininen voisi syödä punaisen ja syödä uudestaan toisen punaisen (siniset nuolet).

Tämä olisi punaisen kannalta huono siirto. Sen takia algoritmi antaa jokaiselle siirrolle pisteen.

Pelatessa toinen yrittää maksimoida pisteet ja toinen pelaaja minimoida pisteet. Siitä tulee nimi minmax-algoritmi.

Lähtötilanteessa algoritmi käy läpi eri vaihtoehtoja. Omat vaihtoehdot(punaiset) ja vastustajan vaihtoehdot (siniset)



Kuvio 5 Esimerkki siirtojen pisteytyksestä

Ohjelma valitsee siirrot, jotka tuottavat maksimaaliset pisteet. Punainen valitsee 2, jonka seurauksena sininen haluaa minimoida mahdolliset pisteet ja valitsee myös 2.

3 Chinook

Chinook on ohjelma, jonka rakentaminen on aloitettu jo vuonna 1988, jonka tarkoituksena oli voittaa ihminen tammipelissä. Chinook saatiin täysin valmiiksi vuonna 2007 Albertan yliopistossa, jolloin ohjelman katsottiin pystyvän sataprosenttisesti tuottamaan pelaajaa vastaan vähintään tasapeli, toisin sanoen, ohjelmaa on mahdollonta voittaa. Chinook on ensimmäinen tietokoneohjelma, joka on voittanut maailmanmestaruuden ihmistä vastaan (vuonna 1994). (Schaeffer, 2007)

3.1 Chinookin historia

1989 Albertan yliopistossa Kanadassa Jonathan Schaeffer alkoi rakentaa ohjelmaa, jonka tarkoituksena oli pelata täydellistä tammea. Myöhemmin työryhmässä olivat mukana myös Rob Lake, Paul Lu, Martin Bryant, and Norman Treloar. (Chinook (computer program), Wikipedia, 2021)

Chinook kasvoi pikkuhiljaa. 1990-luvun alussa tietokoneet eivät olleet niin kehittyneitä, kuin nykypäivänä. Prosessorien tehot, sekä muistit olivat hyvin rajallisia ja ne maksoivat paljon. Työryhmä alkoi rakentamaan tietokantaa, jossa olisi tammen ”kaikki mahdolliset siirrot” resurssien sallimissa rajoissa. Ensimmäinen ”Endgame”-tietokanta koostui kaikista mahdollisista pelitilanteista, jotka olivat mahdollisia, kun pelilaudalla on jäljellä neljä tai vähemmän pelinappuloita (noin 7 miljoonaa). (Schaeffer, 2007)

Taulukko 1 Endgame-tietokannan kasvu 1989–2007

Vuosi	Pelinappulat (n tai vähemmän)	Pelitilanteiden määrä
1989	4	7,092,774
1990	5 ja 6	148,688,232 ja 2,503,611,964
1992	7	34,779,531,480
1994	8	406,309,208,481
2005	10	34,778,882,769,216
Yhteensä		39,271,258,813,439

Työryhmä pääsi testaamaan Chinookia tammipelin parhaimpien kanssa ensimmäisen kerran vuonna 1992, jolloin se hävisi Marion Tinsleylle ja sijoittui toiseksi World Checkers Championshipissä Man-Machine- sarjassa. Chinook voitti maailman mestarin tittelin vuosina 1994, 1995 ja 1996, jonka jälkeen ohjelma ei enää kilpaillut kyseisestä tittelistä ollessaan liian ylivoimainen. (Chinook (computer program), Wikipedia. 2021)

3.2 Chinookin käyttämät algoritmit

Chinook koostuu useasta eri osasta, jotka voidaan karkeasti jakaa kahteen pääkategoriaan: lopputietokanta, sekä pelin laskuosa.

3.2.1 Lopputietokanta (Backward search / endgame database)

Tietokanta pelin mahdollisista lopputilanteista, joissa pelilaudalla on kymmenen tai vähemmän pelinappulaa ($3,9 * 10^{13}$ kappaletta). Eli voidaan sanoa, että kun pelilaudalta on syöty tarpeeksi nappuloita ja jäljellä on enää kymmenen tai alle, voidaan peli "brute forcettaa" loppuun. Jokaiselle pelitilanteelle on laskettu voitto/tasapeli/häviö -arvo. Arvot lasketaan lopputilanteesta "takaperin" lähtötilanteeseen, josta englanninkielinen termi "backward search" tulee (vrt. min-max-algoritmi). Tietokanta on suuri, ja se on pakattu 237 gigatavuun, eli noin 154 pelitilannetta / tavu. Chinook käyttää omia pakkaus- ja purkualgoritmeja, jotka ovat optimoitu ohjelman tietorakenteeseen sopiviksi. (Schaeffer, 2007)

Lopputietokantaan on laskettu myös noin 200kpl yksittäisiä pelitilanteiden arvoja, joissa on yli 10 pelinappulaa jäljellä. Nämä pelitilanteet on valittu tärkeiksi ja kriittisiksi tilanteiksi analysoimalla maailman huippujen pelaamia tammipelejä. (Schaeffer, 2007)

3.2.2 Pelin laskuosa (Forward search)

Pelin alusta tilanteeseen, jossa lopputietokantaa voidaan käyttää, Chinook käyttää ohjelman laskuosaa (Forward search). Laskuosa koostuu kahdesta osasta: totuuspään hallinta ja ratkaisijat. (Schaeffer, 2007)

Totuuspuun hallintaosa (Proof tree manager) sisältää ja ylläpitää pelin totuustaulua. Totuustauluun on tallennettu 19kpl erilaisia kolmen siirron aloitusta, joiden loppusyvyys vaihtelee 31–154 siirron välillä. Aloitus, jota ohjelma lähtee toteuttamaan, arvotaan. Hallintaosa käyttää algoritmia (the Proof Number search) tunnistamaan ja arvioidakseen tulevia pelitilanteita, jotka eivät ole vielä totuustaulussa. Pelitilanteet, jotka se priorisoi, lähetetään ratkaisijoille (Proof tree solvers) laskettaviksi. (Schaeffer, 2007)

Table 2. Openings solved. Shown are the opening moves (using the standard square number scheme in Fig. 1, bottom), the result, the number of positions given to the solvers, and the position farthest from the start of the game that was searched (Max ply). The last two columns give the size and ply depth of the pruned minimal proof tree. Note that the total does not match the sum of the 19 openings. The combined tree has some duplicated nodes, which have been removed when reporting the total.

No.	Opening	Proof	Searches	Max ply	Minimal size	Max ply
1	09-13 22-17 13-22	Draw	736,984	56	275,097	55
2	09-13 21-17 05-09	Draw	1,987,856	154	684,403	85
3	09-13 22-18 10-15	Draw	715,280	103	265,745	58
4	09-13 23-18 05-09	Draw	671,948	119	274,376	94
5	09-13-23-19 11-16	Draw	964,193	85	358,544	71
6	09-13 24-19 11-15	Draw	554,265	53	212,217	49
7	09-13 24-20 11-15	Draw	1,058,328	59	339,562	58
8	09-14 23-18 14-23	≤Draw	2,202,533	77	573,735	75
9	10-14 23-18 14-23	≤Draw	1,296,790	58	336,175	55
10	10-15 22-18 15-22	≤Draw	543,603	60	104,882	41
11	11-15 22-18 15-22	≤Draw	919,594	67	301,310	59
12	11-16 23-19 16-23	≤Draw	1,969,641	69	565,202	64
13	12-16 24-19 09-13	Loss	205,385	44	49,593	40
14	12-16 24-19 09-14	≤Draw	61,279	45	23,396	44
15	12-16 24-19 10-14	≤Draw	21,328	31	8,917	31
16	12-16 24-19 10-15	≤Draw	31,473	35	13,465	35
17	12-16 24-19 11-15	≤Draw	23,803	34	9,730	34
18	12-16 24-19 16-20	≤Draw	283,353	49	113,210	49
19	12-16 24-19 08-12	≤Draw	266,924	49	107,109	49
Overall		Draw	Total 15,123,711	Max 154	Total 3,301,807	Max 94

Kuvio 6 Tallennetut aloitukset

Ratkaisijat suorittavat hallintaosan antamien pelitilanteiden laskemiseen kahta algoritmia: alpha-beta ja Df-pn-algoritmit. Alpha-beta:n suoritusaika on määritetty 15 sekuntiin, jonka aikana se laskee 17–23 siirtoa eteenpäin. Jos alpha-beta ei tuota annetussa ajassa arvoa, Df-pn-algoritmi otetaan käyttöön. Df-pn-tuottaa samankaltaisen totuuspuun, kuin alpha-beta, mutta se ei ole niin tarkka. Algoritmit tuottavat pelitilanteille kolmenlaisia arvoja: suora lopputulos (voitto/häviö/tasapeli), osittain ratkaistu (vähintään tasapeli / enintään tasapeli) tai heuristinen (kuinka hyvä tai huono pelitilanne on). Tulokset välitetään takaisin hallintaosalle, joka tekee päätöksen seuraavista siirroista niiden perusteella. (Schaeffer, 2007)

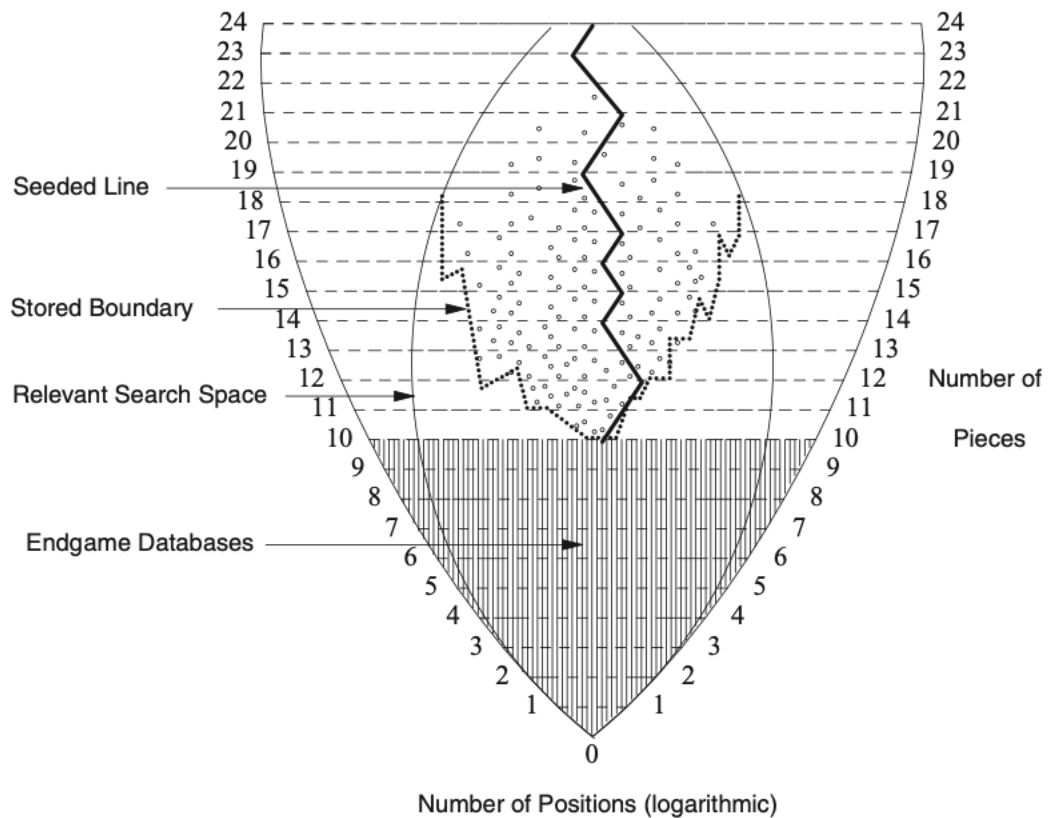
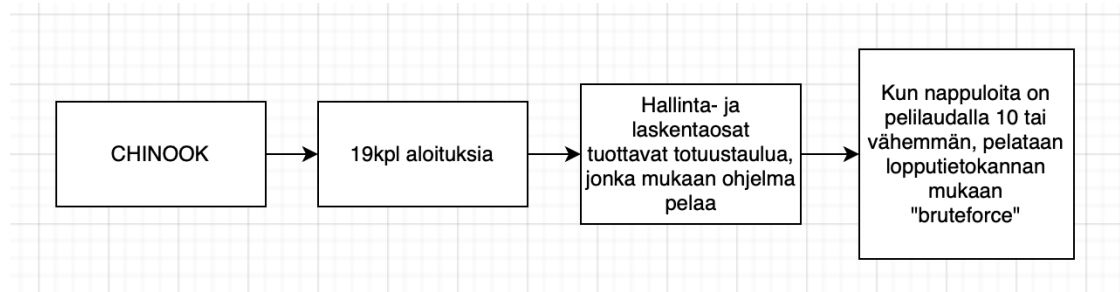


Fig. 2. Forward and backward search. The number of pieces on the board are plotted (vertically) versus the logarithm of the number of positions (Table 1). The shaded area shows the endgame database part of the proof—i.e., all positions with ≤ 10 pieces. The inner oval area shows that only a portion of the search space is relevant to the proof. Positions may be irrelevant because they are unreachable or are not required for the proof. The small open circles indicate positions with more than 10 pieces for which a value has been proven by a solver. The dotted line shows the boundary between the top of the proof tree that the manager sees (and stores on disk) and the parts that are computed by the solvers (and are not saved in order to reduce disk storage needs). The solid seeded line shows a “best” sequence of moves.

3.3 Chinookin toiminta kuvana



Kuvio 8 Chinookin toiminta lyhyesti kuvana

4 Algoritmien vertailu

4.1 Harjoitustyössä käytetty checkers AI

Käytimme tähän harjoitustyöhön valmista tammipelin min-max-AI-pakettia, jonka löysimme GitHubista. Koodin on tehnyt yhdysvaltalainen Tim Ruscica ja se on löydettävissä osoitteessa: <https://github.com/techwithtim>.

Minmax-algoritmin tehokkuus riippuu oleellisesti siitä, kuinka monen siirron päähän ohjelma laskee. Algoritmi on rekursiivinen ja päätöspuu kasvaa nopeasti hyvin suureksi, joka vaatii muistia. Tammipelissä päätöspuun leveys riippuu vuoron mahdollisista siirroista, joihin vaikuttavat pelin säännöt.

Testasimme min-max algoritmin käyttämää aikaa muuttamalla numeroarvoa, joka määrittää, kuinka monen siirron päähän algoritmi laskee. Numerolla 4 pelin siirto kesti noin 5 sekuntia ja numerolla 5 noin 42 sekuntia. Numerolla 6 aika oli 5 min 33 sekuntia.

```
def main():
    run = True
    clock = pygame.time.Clock()
    game = Game(WIN)

    while run:
        clock.tick(FPS)

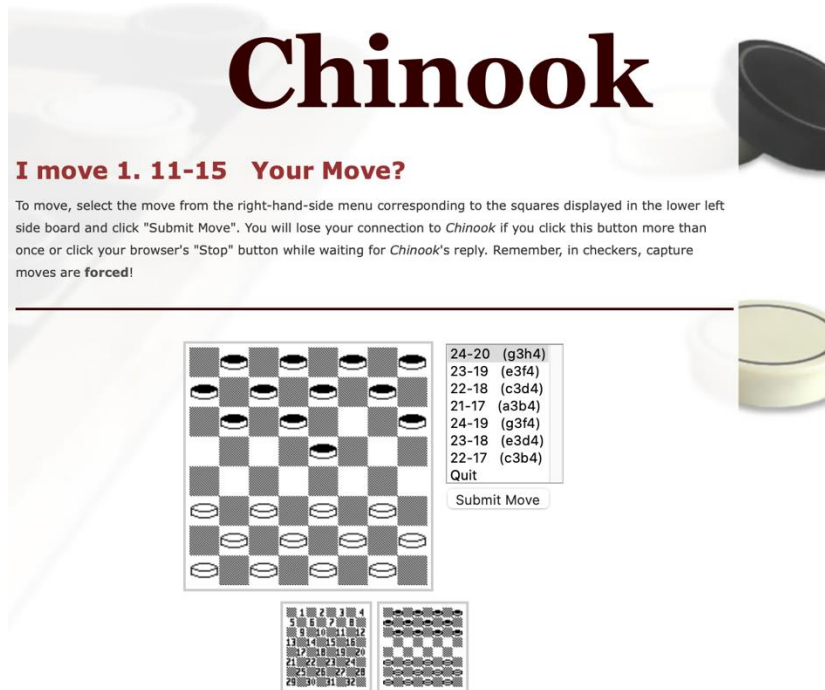
        if game.turn == WHITE:
            value, new_board = minimax(game.get_board(), 5, WHITE, game)
            game.ai_move(new_board)
```

Kuvio 9 min-max-muutos

4.2 Min-max vastaan Chinook

Min-max-algoritmin ja Chinook-ohjelman vertailu on hieman epäreilua, koska yhden algoritmin ja juuri kyseiseen ”ongelmaan” tehdyn kokonaisen ohjelman välinen ero on niin suuri. Min-max-algoritmi (alpha-beta) on yksi Chinookin ”osa” ja halusimme testata, kuinka nämä kaksi pärjäisivät pelatessaan vastakkain.

Testasimme GitHubista löytämäämme checkers-AI:ta ja Chinookia vastakkain niin, että checkers AI-pythonpakettia ajettiin työasemalla Visio Studio Codella ja Chinook Albertan yliopiston sivuilta selaimessa. Teimme niin, että Chinook aloitti ja matkimme siirron Visual Studio Coden peliin itse manuaalisesti. Checkers AI:n tuottama tulos taas matkittiin selaimen ja näin pelasimme pelin loppuun asti. Tekoälyjen väliseen peliin meni huomattavasti aikaa, koska Chinook ei toiminut paikallisesti, vaan Albertan yliopiston palvelimessa ja aikaviive siirtojen välillä oli aika pitkä. Lopputuloksena Chinook kuitenkin voitti odotetusti.



Kuvio 10 Chinookin käyttöliittymä

5 Yhteenveto / pohdinta

Valitsimme aiheen, koska se vaikutti mielenkiintoiselta. Halusimme kokeilla pelin demoamista sekä pelin kulun laskemista. Aihe tuntui ensin helpolta, mutta osoittautui osittain vaikeaksi; tammipelistä oli vähän suomenkielistä tietoa, joten jouduimme käyttämään englannin kielistä materiaalia hyödyksi ja suomentamaan löytämämme materiaalit. Suomentamisessa meni jo oman aikansa sekä demon tekemisessä, aikataulu oli myös tiukka koska omaa aikaamme meni myös muiden kurssien tehtävien ja projektien parissa. Saimme kuitenkin ryhmässä projektin tekemisen aikatauluksen hyvin sopimaan omiin menoihimme.

Opimme demoamaan pelejä sekä opimme kuinka min-max algoritmit toimivat.

Emme olleet koskaan ennen tehneet pelin demoamista, joten halusimme kokeilla ja tehdä tämän projektin. Demoaminen onnistui mielestämme ihan hyvin, löysimme valmiin koodin ja kokeilimme, muokkailimme sekä kommentoimme koodia, että osoitamme että tiedämme mitä koodi tekee tietyissä tilanteissa.

Aikaa ryhmältämme meni noin 20 h kokonaisuudessaan, suurin osa ajasta meni raportin kirjoittamisen sekä lähdeaineiston etsimisen parissa.

Ongelmakohtia työssämme oli lähdemateriaalin löytäminen ja sen käyttö. Materiaalia löytyi, mutta suurimmaksi osaksi eri nettisivuilla oli samaa materiaalia eri sanoin käytetty hyödyksi. Yritimme ottaa tärkeimmän asiat huomioon omaan raporttiimme. Ongelmana oli myös tiukka aikataulu, käytimme viimeiset päivät suurimmaksi osaksi koneen ääressä, että saamme työn valmiiksi; meillä oli monta muuta tehtävää samaan aikaan.

Yllätyimme, kuinka aikaa vievää ja työlästä projektimme raportointi ja demoaminen lopulta oli. Myös muut kurssit söivät aikaa tältä projektilta; olisi ollut hyvä, että aikaa olisi ollut enemmän.

Lähteet

Alexander Katz, Eli Ross. Brilliant wiki. Viitattu 7.12.2021

<https://brilliant.org/wiki/minimax/>

American Psychological Association, Game theory. Viitattu 7.12.2021.

<https://dictionary.apa.org/minimax-strategy>

Janet Chen, Su-I Lu ja Dan Vekhter. Stanford, Game theory. Viitattu 7.12.2021.

<https://cs.stanford.edu/people/eroberts/courses/soco/projects/1998-99/game-theory/Minimax.html>

Jonathan Schaeffer, Checkers Is Solved. Viitattu 7.12.2021

https://www.cs.mcgill.ca/~dprecup/courses/AI/Materials/checkers_is_solved.pdf

Tim Ruscica, 2020. Tech With Tim. Python Checkers AI. Päivitetty 1. Syyskuu 2020

<https://github.com/techwithtim/Python-Checkers-AI>

<https://github.com/techwithtim/Python-Checkers>

University of Alberta, 2011. World Man-Machine Checkers Champions. Viitattu 7.12.2021.

<http://webdocs.cs.ualberta.ca/~chinook/>

John, D. Hedengren. 2020. Minimax and Maximin Optimatizion, Apmonitor.

Viitattu 7.12.2021 <http://apmonitor.com/me575/index.php/Main/MiniMax>

Wikipedia, Chinook (computer program). Viitattu 7.12.2021

[https://en.wikipedia.org/wiki/Chinook_\(computer_program\)](https://en.wikipedia.org/wiki/Chinook_(computer_program))

Wikipedia, Tammi (peli). Viitattu 7.12.2021

[https://fi.wikipedia.org/wiki/Tammi_\(peli\)](https://fi.wikipedia.org/wiki/Tammi_(peli))